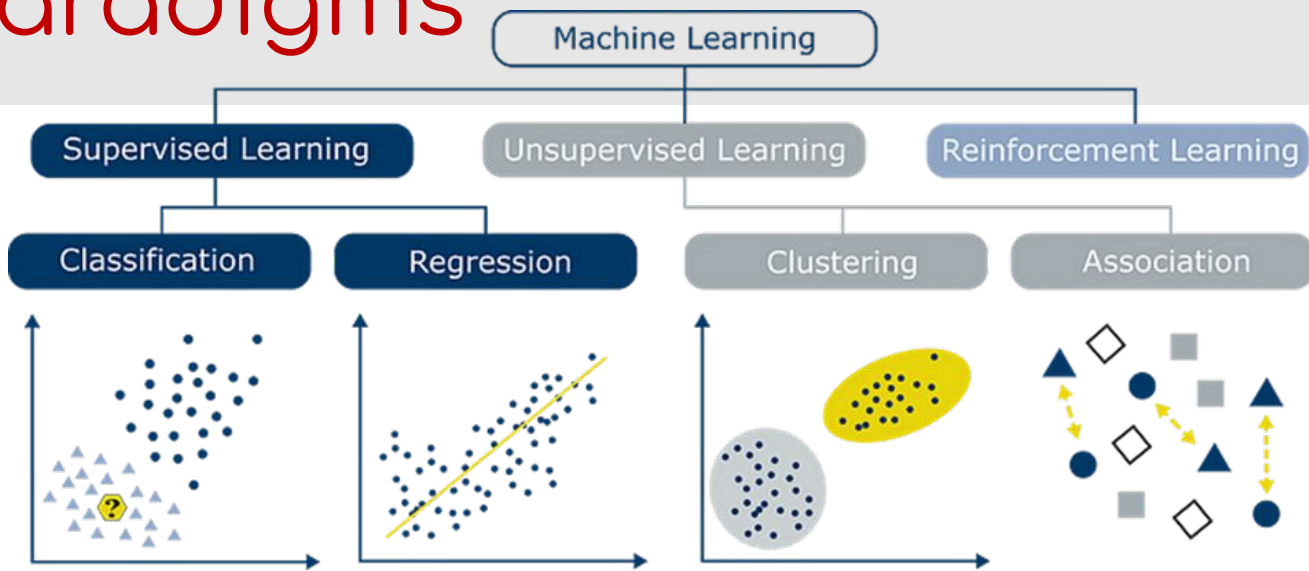


Computer vision

Object detection & Object recognition

Basics of machine learning - recap.?

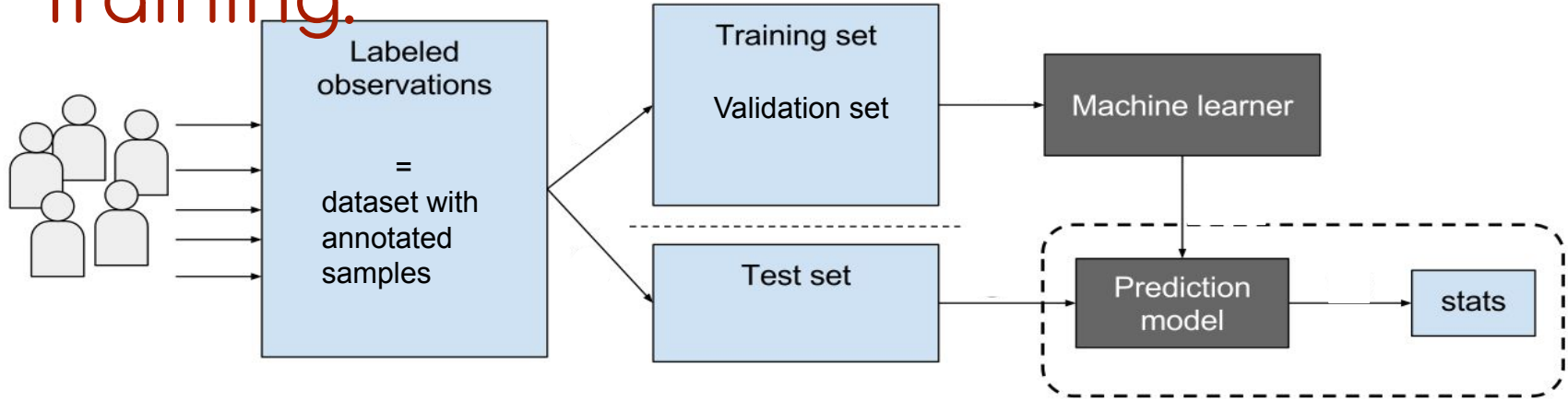
Learning Paradigms



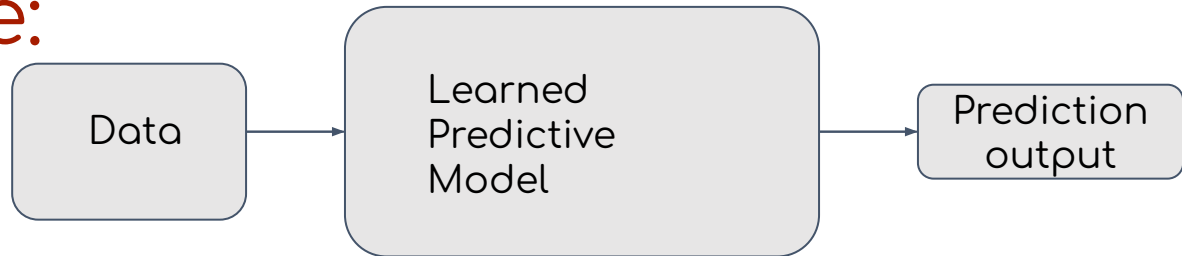
- **Supervised learning**
fully supervised, semi-supervised, weakly supervised and more...
- **Unsupervised learning**
fully unsupervised learning, semi-supervised
- **Reinforcement learning**

Supervised learning - basic concept

Training:



Inference:



Types of supervision

Fully supervision

usage of a **fully labeled** dataset to learn the relationship between input and output data

Weak supervision

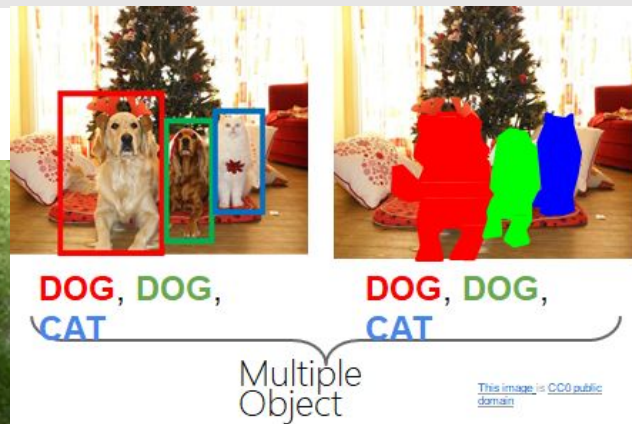
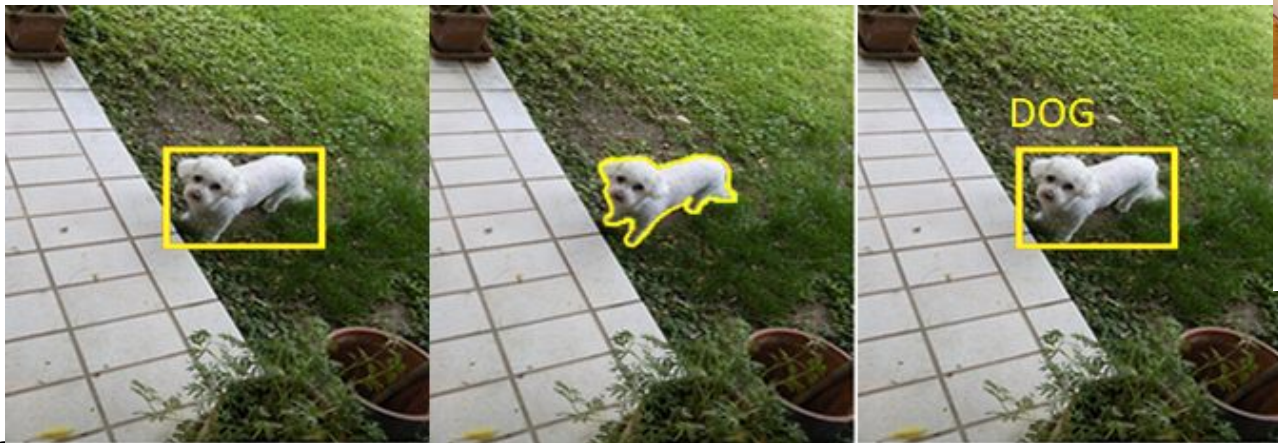
training with labels that may include **mistakes in label annotations**

Semi-supervised

combines partially supervised and partially unsupervised approach by using a **small amount of** labeled data alongside a large amount of unlabeled data to train a model

Object recognition

Type of task



Object detection Object segmentation Object detection and recognition

Example: people detection vs. people recognition, verification



People detection

-> identifying and localizing human subjects within an image, typically represented by bounding boxes around detected individuals



Person recognition

→ Identifying a person's identity from a dataset.

Person verification

→ Confirming whether a claimed identity is correct (Yes/No).

Challenges in object recognition

The recognition method should be invariant to changes in:

scale,
orientation,
illumination,
camera viewpoint,
occlusion,
object pose,
intra-class variations...



Used methods used for object detection & object recognition

Pattern matching

- Rule based pattern recognition

- Statistical pattern recognition

- Fuzzy pattern recognition

Artificial intelligence -> typically classification task

Feature extraction + classification

Neural networks

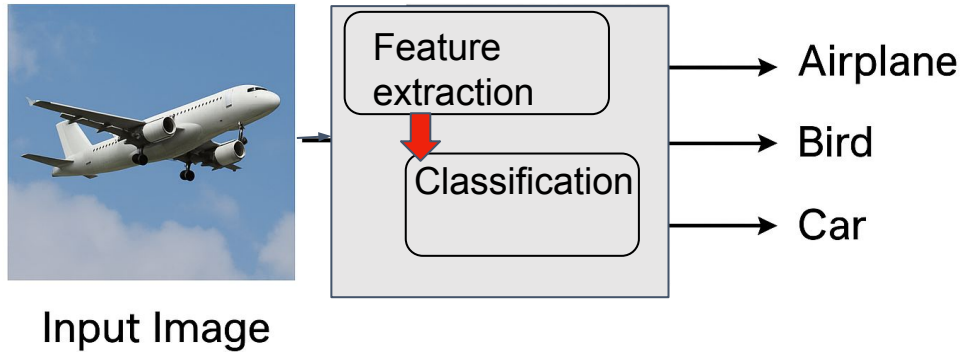
- Methods of Deep learning

 - Convolutional neural networks CNN

 - Vision transformers

Object recognition as a classification task

Traditional Approach of Object Recognition



The task of object recognition within an image is treated as a classification problem.

The process is typically divided into two main stages: feature extraction and classification.

Feature extraction

Feature extraction stage

The goal is to transform raw image data into a set of discriminative features that effectively describe the object's visual characteristics.

In the traditional computer vision pipeline, feature extraction plays a crucial role in transforming raw pixel data into a more meaningful representation that can be used for object recognition or classification.

Features aims to describe the object by description of:

- Texture
- Color
- Shape

Examples:

Color histograms , Gabor filters, HOG (Histogram of Oriented Gradients)



Input Image

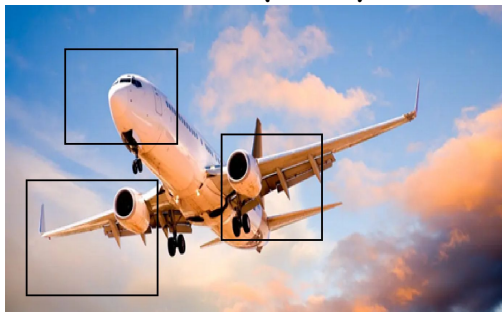
$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

Feature vector

Feature extraction

Input:

Image, Region of Interest (ROI), or a tile/patch of the image



Output:

Feature vector or feature matrix



Input Image

↓

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

Feature vector

Feature extraction using convolution

Handcrafted (Manually Designed) Filters

One common approach of feature extraction involves using convolution operations with manually designed (handcrafted) filter kernels.

Feature extraction using convolution is based on the principle that important visual characteristics — such as edges, corners, and textures — can be detected using filtration by various filters.

Before deep learning, filter kernels were not learned from data. Instead, computer vision experts **manually designed filters based on known mathematical and perceptual properties to detect specific types of features.**

Some common handcrafted filters include:

- Sobel / Prewitt Detect horizontal or vertical edges
- Laplacian Detect edges and fine details (second-order derivative)
- Gabor Filter Capture oriented textures and frequency patterns
- Tunable sinusoidal kernel
- Gaussian Filter Perform smoothing and reduce noise
- Haar-like Features Detect contrast-based patterns (used in face detection)

Classification

Classification stage

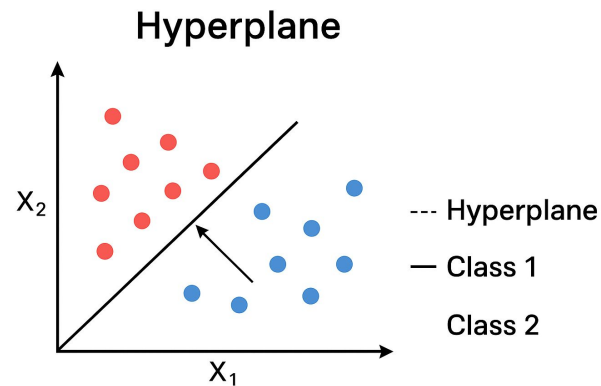


The extracted features are passed into a classifier that decides which class the object belongs to.

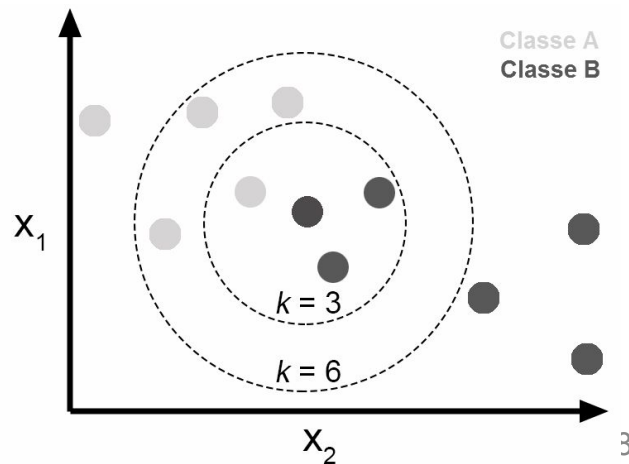
A machine learning algorithm like Support Vector Machine (SVM), k-NN, Random Forest, Bayes Classifier, MLP or other is trained using labeled feature vectors derived from data.

Classifier examples

Simple Linear Classifier

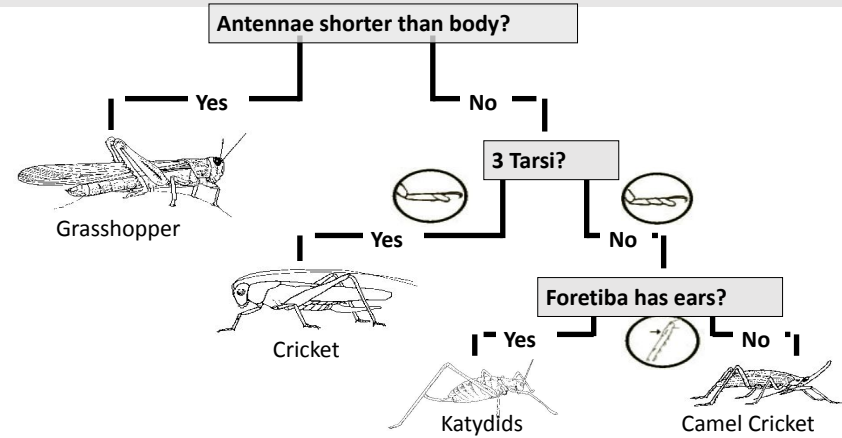


k-nearest neighbours (k-NN)



Classifier examples

Decision Tree Classifier



The Random Forests

Given a training set S .
for $i = 1$ to k :

Build **subset** S_i by sampling with replacement from S , learn tree T_i from S_i

Make predictions according to majority vote of the set of k trees.



k : number of trees T

Classifier examples

AdaBoost Algorithm

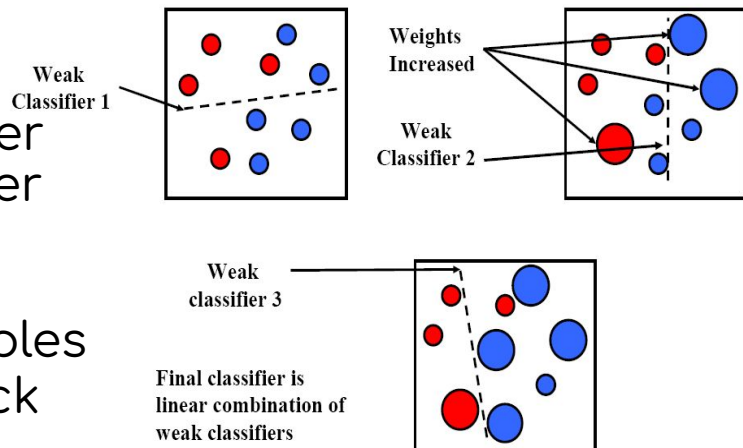
Build a strong classifier by combining number of “weak classifiers”, which need only be better than 50%

Start with uniform weights on training examples
Evaluate weighted error for each feature, pick best.

Incorrectly classified -> more weight

Correctly classified -> less weight

Final classifier is linear combination of the weak ones, weighted according to error they had.



Classifier examples

Support vector machine SVM

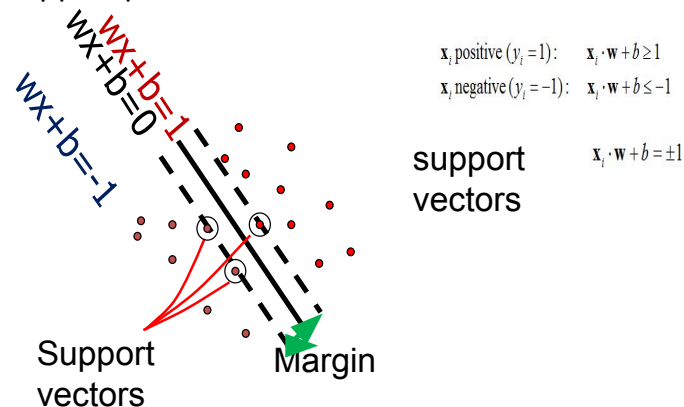
A support vector machine constructs a hyperplane or set of hyperplanes in a high-dimensional space, which can be used for classification, regression or other tasks.

Consider a binary classification problem $(-1, 1)$

Bayes classifier

is a probabilistic classifier based on applying Bayes' theorem (from Bayesian statistics)

hyperplane that maximizes the margin



$$p(h | d) = \frac{P(d | h) P(h)}{P(d)}$$

$P(h)$:

prior belief (probability of hypothesis h before seeing any data)

$P(d | h)$:

likelihood (probability of the data if the hypothesis h is true)

$P(d) = \sum_h P(d | h) P(h)$:

data evidence (marginal probability of the data)

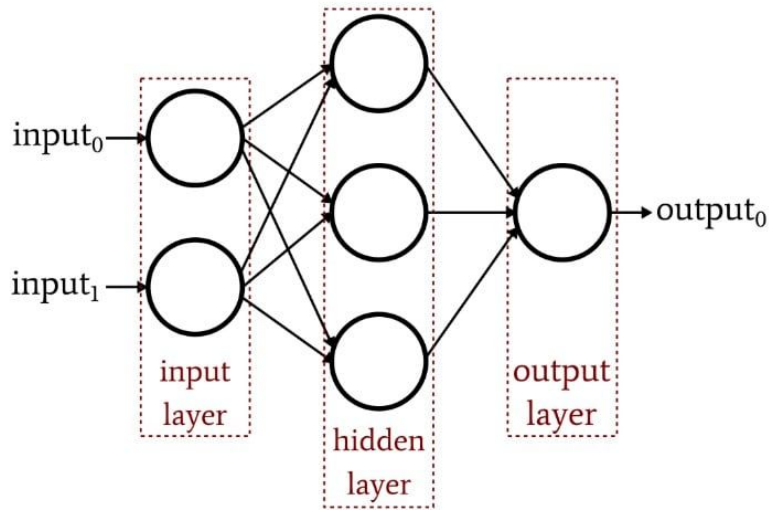
$P(h | d)$:

posterior (probability of hypothesis h after having seen the data d)

Classifier examples

MLP

When applied to classification, the MLP learns to map input feature vectors to specific class labels by discovering complex nonlinear decision boundaries through multiple layers of interconnected neurons.

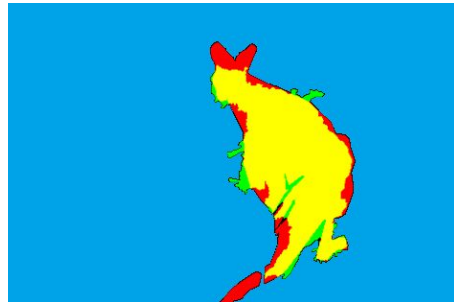


Evaluation of classification

true positive (TP), true negative (TN) false positive (FP), false negative (FN)

groundTruth:

segmentation result:



Task: Object/background segmentation

TP : Number of true predicted pixels from all real positive cases in the data (yellow)

TN : Number of true predicted pixels from all real negative cases in the data (blue)

FP : Number of false predicted pixels from all real positive cases in the data (red)

FN : Number of false predicted pixels from all real negative cases in the data (green)

Precision and recall

precision or positive predictive value (PPV)

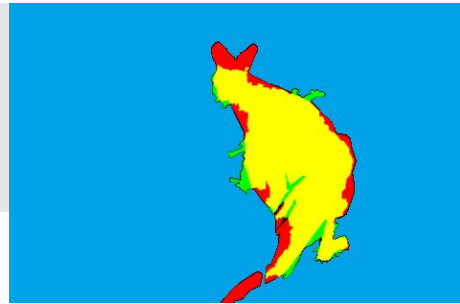
$$PPV = \frac{TP}{TP + FP}$$

Precision is the fraction retrieved that are relevant to the user's information need.

recall, sensitivity, hit rate, or true positive rate (TPR)

$$TPR = \frac{TP}{TP + FN}$$

Recall is the fraction that are relevant to the query that are successfully retrieved.



TP : Number of true prediction from all real positive cases in the data (**yellow**)

TN : Number of true prediction from all real negative cases in the data (**blue**)

FP : Number of false prediction from all real positive cases in the data (**red**)

FN : Number of false prediction from all real negative cases in the data (**green**)

Accuracy (ACC), F1 score

Accuracy (ACC)

$$ACC = \frac{TP + TN}{P + N}$$

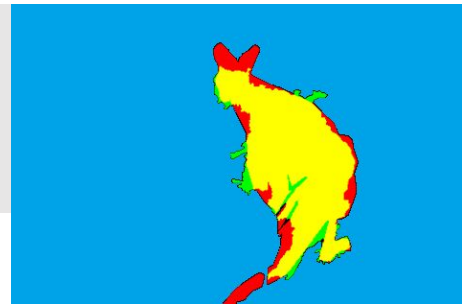
$$P = TP + FN$$

$$N = FP + TN$$

The F1 score is the harmonic mean of the precision and recall.

$$F_1 = 2 * \frac{\text{precision} * \text{recall}}{\text{precision} + \text{recall}}$$

$$F_1 = \frac{2TP}{2TP + FP + FN}$$



TP : Number of true prediction from all real positive cases in the data (**yellow**)

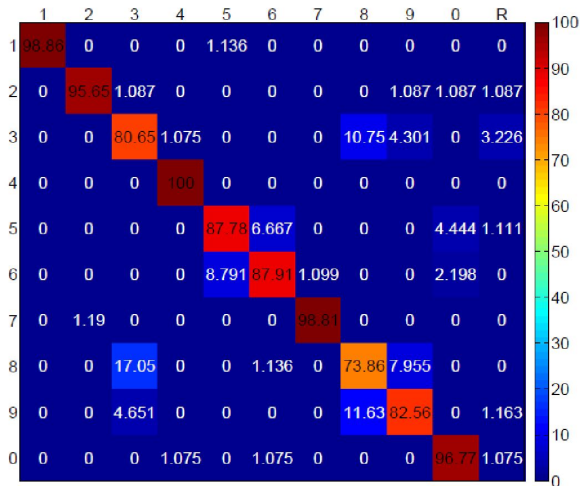
TN : Number of true prediction from all real negative cases in the data (**blue**)

FP : Number of false prediction from all real positive cases in the data (**red**)

FN : Number of false prediction from all real negative cases in the data (**green**)

Confusion matrix - error matrix (Matica zámén)

Each column of the matrix represents the instances in a predicted class while each row represents the instances in an actual class (or vice versa).



Binary classification:

		Observation	
		Yes	No
Prediction	Yes	a true positive hit success	b false positive false alarm Type I error
	No	c false negative miss Type II error	d true negative correct negative correct rejection

Receiver Operator Characteristic - ROC

Receiver Operator Characteristic - ROC

Plot “True Positive Rate TPR” vs. “False Positive Rate FPR”

$$\text{TPR} = \text{TP} / \text{P}$$

$$\text{P} = \text{TP} + \text{FN} \Rightarrow \text{TPR} = \text{TP} / (\text{TP} + \text{FN})$$

Also called hit rate, recall, sensitivity

$$\text{FPR} = \text{FP} / \text{N}$$

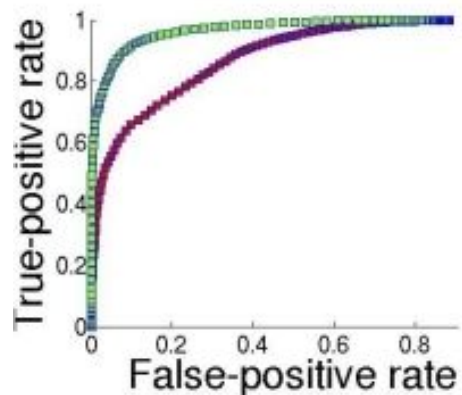
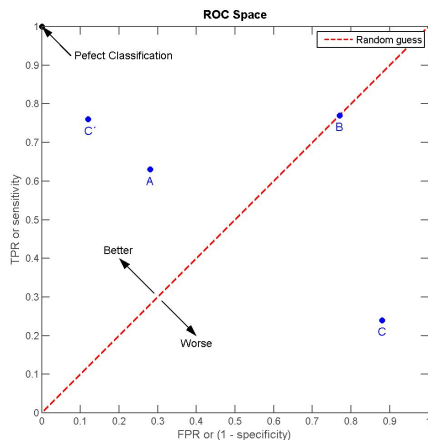
$$\text{N} = \text{FP} + \text{TN} \Rightarrow \text{FPR} = \text{FP} / (\text{FP} + \text{TN})$$

Also called false alarm rate, fall-out, =1-specificity

Receiver Operating Characteristic - ROC

Receiver Operating Characteristic curve, or ROC curve, is a graphical plot that illustrates the performance of a binary classifier system as its **discrimination threshold** is varied.

The curve is created by plotting the true positive rate (TPR) against the false positive rate (FPR) at various threshold settings.

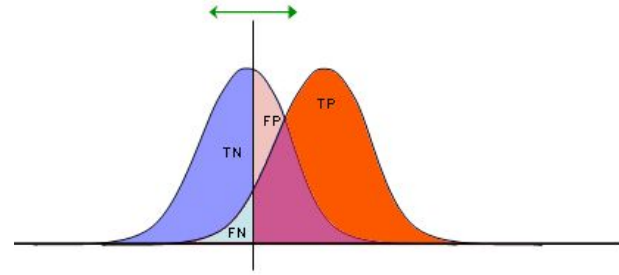


ROC curve example

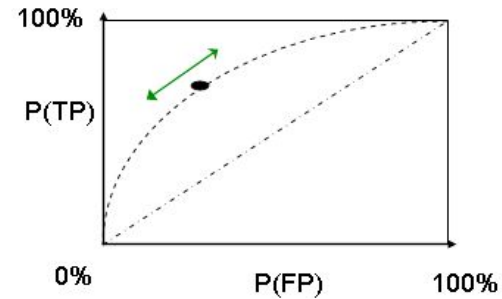
ROC Curve, Area Under Curve (AUC)

AUC stands for "Area under the ROC Curve."

AUC measures the entire two-dimensional area underneath the entire ROC curve (think integral calculus) from (0,0) to (1,1).



TP	FP
FN	TN
1	1



Next parameters for evaluation of classifier

- Time consumption:
 - Time to **construct** the classifier
 - Time to **use** the model (run time)
- Scalability
- Robustness
- Noise
- Missing values

- Interpretability (if possible)

Basic approaches of object detection and object recognition

Basic approaches of object detection and object recognition

Traditional approach based on classification task:

- Global approach : Sliding Windows
- Local : Local descriptors

Era of Deep Learning:

- Convolutional Neural Network (CNN)
- Vision Transformer

Global approach

Global Appearance & Sliding Windows

Sliding window

Size of the window?

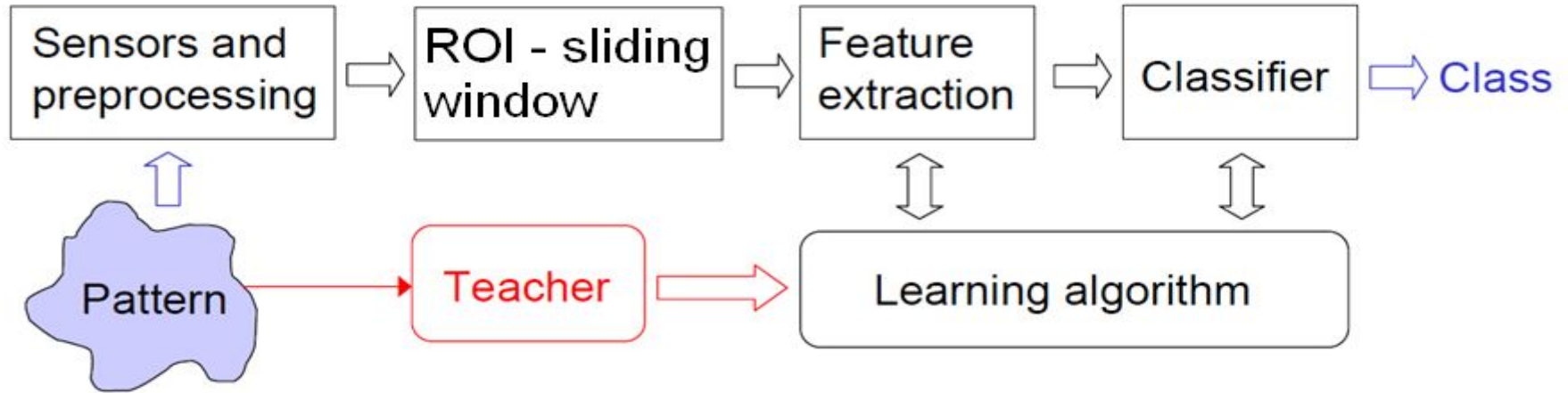
Orientation of the window?



Examples: Face detection,
People detection,



Global Appearance & Sliding Windows



ROI: Region of Interest

Global Appearance & Sliding Windows

Object detection

For object detection, the classification task is a binary decision.

The question that answers classifier:

„Is in the given window the object? (yes or no?)“

! Important is choice of calculated features and also important is choice of a classifier!

Global Appearance & Object recognition

ROI Selection:

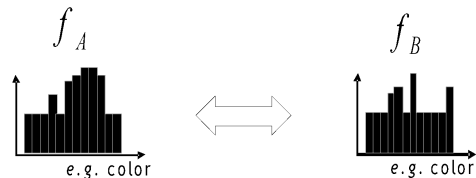
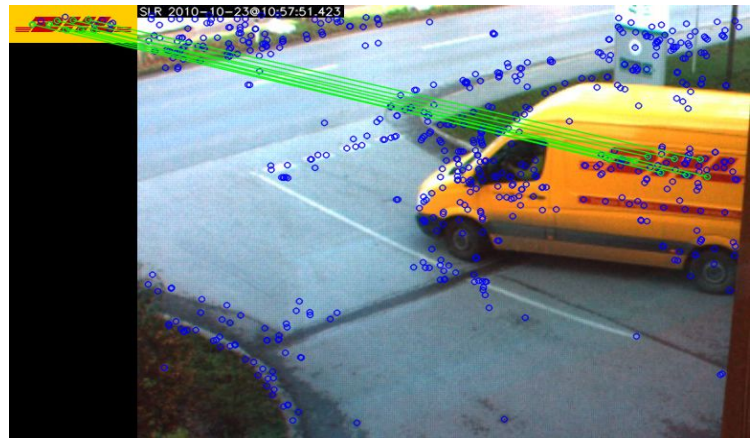
1. The Region of Interest (ROI) is first proposed or identified. It can be manually selected (e.g., bounding box drawn by a user), or automatically proposed using algorithms such as:
 - a. Sliding window
 - b. Selective Search : Edge boxes[1], methods using segmentation, Bounding box proposals and more..
2. Feature extraction and classification of the object (class)

[1]ZITNICK, C. Lawrence; DOLLÁR, Piotr. Edge boxes: Locating object proposals from edges. In: *European conference on computer vision*. Cham: Springer International Publishing, 2014. S. 391-405.

Local : Local descriptors

Object detection using local descriptors

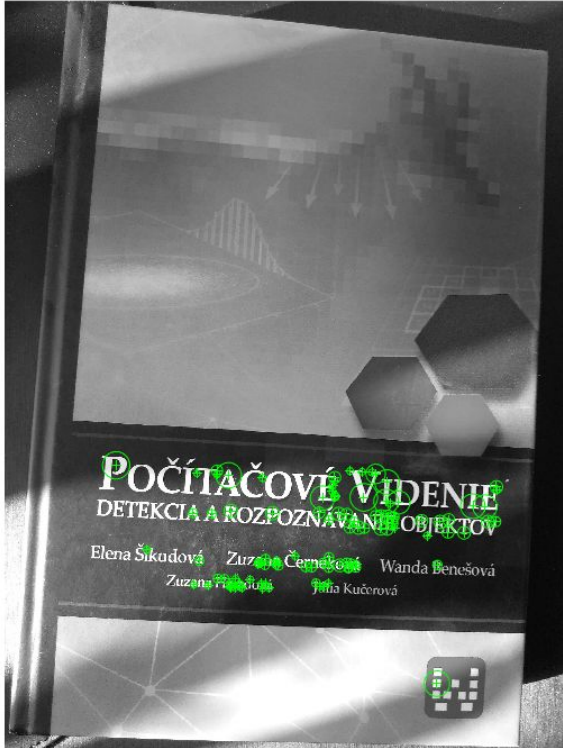
1. Find distinctive key-points using key-point detectors and define a region around each keypoint
2. Compute a local description for the normalized region using key-point descriptors
3. Match the local descriptors
4. Filter the of inlier/outlier
5. Calculate homography



$$d(f_A, f_B) < T$$

Key points detection and description

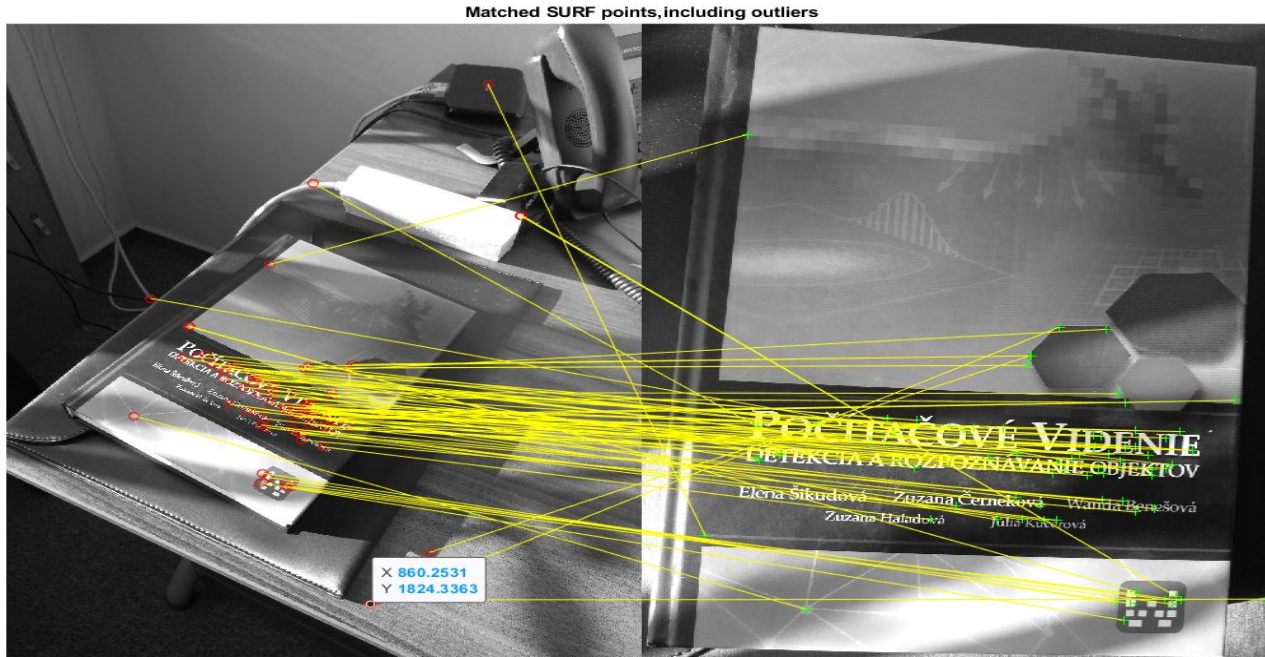
100 Strongest Feature Points from Box Image



300 Strongest Feature Points from Scene Image



Match local descriptors



Filtering of inlier/outlier

Matched inlier points



6. Calculate homography

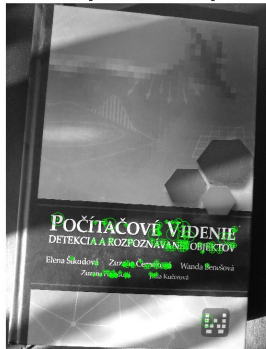
Result after homography transform

Visualisation:

Template projection – blue channel

Original Image – red channel

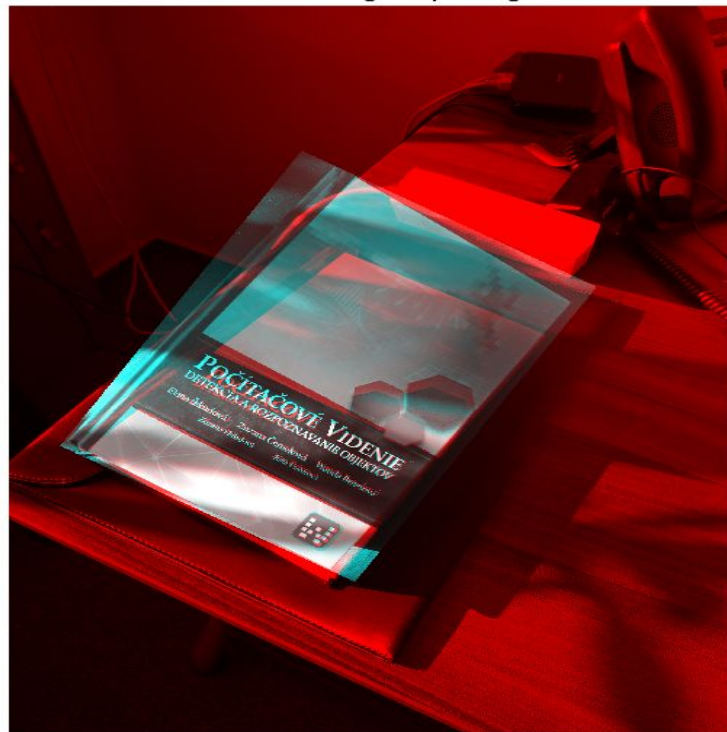
100 Strongest Feature Points from Box Image



300 Strongest Feature Points from Scene Image



Recovered image + Input Image



Local descriptors

More robust than global approach, better from the point of view for:

- occlusions of objects
- changes of camera view position
- rotation, scale invariance
- intra category variations